

Chapter 5: Architecture and Process Model of the RS-DRL-Based Self-Adaptive System

Thesis Documentation

January 5, 2026

1 Chapter Overview and Architectural Perspective

Self-adaptive systems operating under uncertainty require not only effective runtime decision making but also clear architectural separation between monitoring, learning, decision, verification, and execution concerns. While the reference RS-DRL architecture introduces these elements at a high level, its original representation does not explicitly capture the internal process logic and control flow governing their interaction.

This chapter presents a **systematic process-oriented refinement** of the RS-DRL-based approach, defining **the RS-DRL adaptation process** as the dominant process that governs system behavior. Everything else in this chapter is a refinement of that process. This chapter is structured around the runtime adaptation process, not around software components. The primary organizing principle is **process progression**—each section answers the question: “What happens next in the system, and why?” MAPE-K and RS-DRL serve as **explanatory lenses** and **secondary views**, not competing organizational structures. The chapter has two explicit goals: (1) to explain the runtime adaptation process, showing how runtime data is transformed into adaptation decisions, and (2) to explain how RS-DRL integrates into the MAPE-K framework, detailing how learning, verification, and control are orchestrated.

The chapter is organized as follows. Section 2 presents the system-level self-adaptation loop, providing a black-box view of the continuous adaptation process. Section 3 maps the architecture to MAPE-K, establishing the conceptual grounding. Section 4 details the online adaptation decision process, covering monitoring, analysis, planning, verification, and execution. Section 5 presents the learning and policy improvement process. Section 6 explains knowledge management in the RS-DRL architecture. Section 7 discusses the interaction between online decision-making and learning. Finally, Section 8 provides a summary of architectural processes.

To provide navigation and orientation for the process models presented in this chapter, Figure 1 introduces a diagram map that organizes all process diagrams. The map shows the relationship between the central integration diagram and the five detailed activity diagrams that refine individual subprocesses.

Diagram Map of RS-DRL Process Models

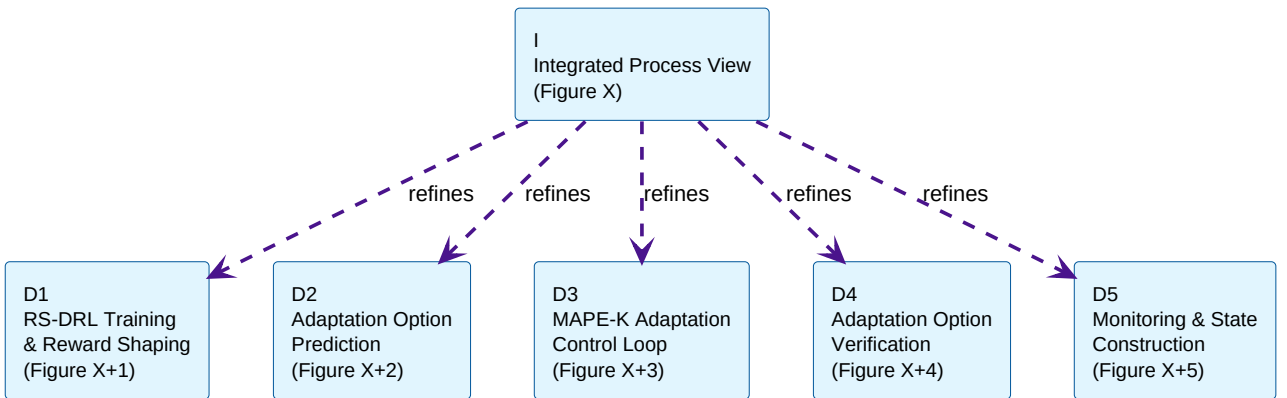


Figure 1: Diagram map of the RS-DRL-based self-adaptive system. The central integration diagram provides a high-level process view, while Diagrams D1–D5 refine the learning, decision-making, verification, control, and monitoring subprocesses presented in this chapter.

Figure 1 serves as a navigation aid for the process models presented in this chapter. The central integration diagram represents the overall runtime behavior of the system, capturing the closed-loop interaction between monitoring, analysis, planning, verification, learning, and execution. Each surrounding diagram refines a specific concern, enabling readers to navigate from high-level overview to detailed subprocess descriptions.

Each surrounding diagram refines a specific process:

- **D1** refines the **Learning Process** (RS-DRL training and reward shaping).
- **D2** refines the **Decision-Making Process** (runtime adaptation option prediction).
- **D3** refines the **Control & Orchestration Process** (MAPE-K adaptation control logic).
- **D4** refines the **Uncertainty Evaluation Process** (adaptation option verification using runtime models).
- **D5** refines the **Perception Process** (monitoring and state construction).

This hierarchical organization ensures that complexity is managed through progressive refinement rather than monolithic representation.

2 System-Level Self-Adaptation Loop

This section presents the system-level self-adaptation loop, providing a black-box view of how the RS-DRL-based self-adaptive system continuously adapts a managed system under uncertainty using learned adaptation policies. The system consists of two main parts: the **Managed System** and the **Managing System**. The Managed System executes the application logic and generates runtime data through its normal operation. The Managing System implements the adaptation logic using the MAPE-K (Monitor, Analyze, Plan, Execute, Knowledge) architectural pattern, enhanced with an RS-DRL module that provides learned intelligence for adaptation decisions.

At the highest level, the system performs one dominant process: continuously adapt a managed system under uncertainty using learned adaptation policies. This process decomposes into a high-level Sense–Plan–Adapt loop: (1) Sense the environment and system state, (2) Plan adaptation decisions using learned policies, and (3) Adapt by executing selected actions. The RS-DRL module is integrated as an upper layer above the MAPE-K loop and is responsible for the continual refinement of the deep reinforcement learning (DRL) model.

The following nine stages describe the runtime adaptation loop; learning episodes are described separately in Section 5 and are shown here only as background updates to Knowledge. Stages 2–3 form an asynchronous learning process that runs in parallel with the runtime adaptation loop (Stages 1, 4–9). Model training occurs asynchronously and updates Knowledge between episodes. Training is conducted over a predefined number of episodes. At the beginning of each episode, the Monitor component of the MAPE-K loop collects the required runtime data (Stage 1) and forwards them to the RS-DRL module (Stage 2). Based on the received input vectors, the RS-DRL module initiates the training process. Upon completion of training, the updated model is stored in the Knowledge repository (Stage 3). Stage 3 (asynchronous): Model update occurs independently of the runtime adaptation cycle. Meanwhile, the Monitor forwards the collected runtime data to the Analyzer for further evaluation (Stage 4). When the Analyzer detects that adaptation goals are violated or are no longer achievable, it notifies the Adaptation Option Predictor, which is embedded in the Planner (Stage 5). The Adaptation Option Predictor retrieves the trained model (Stage 6) and predicts an appropriate adaptation option (Stage 7). The Executor then retrieves the selected option from the adaptation options repository (Stage 8) and applies it to the managed system (Stage 9). This process is repeated for each subsequent adaptation episode.

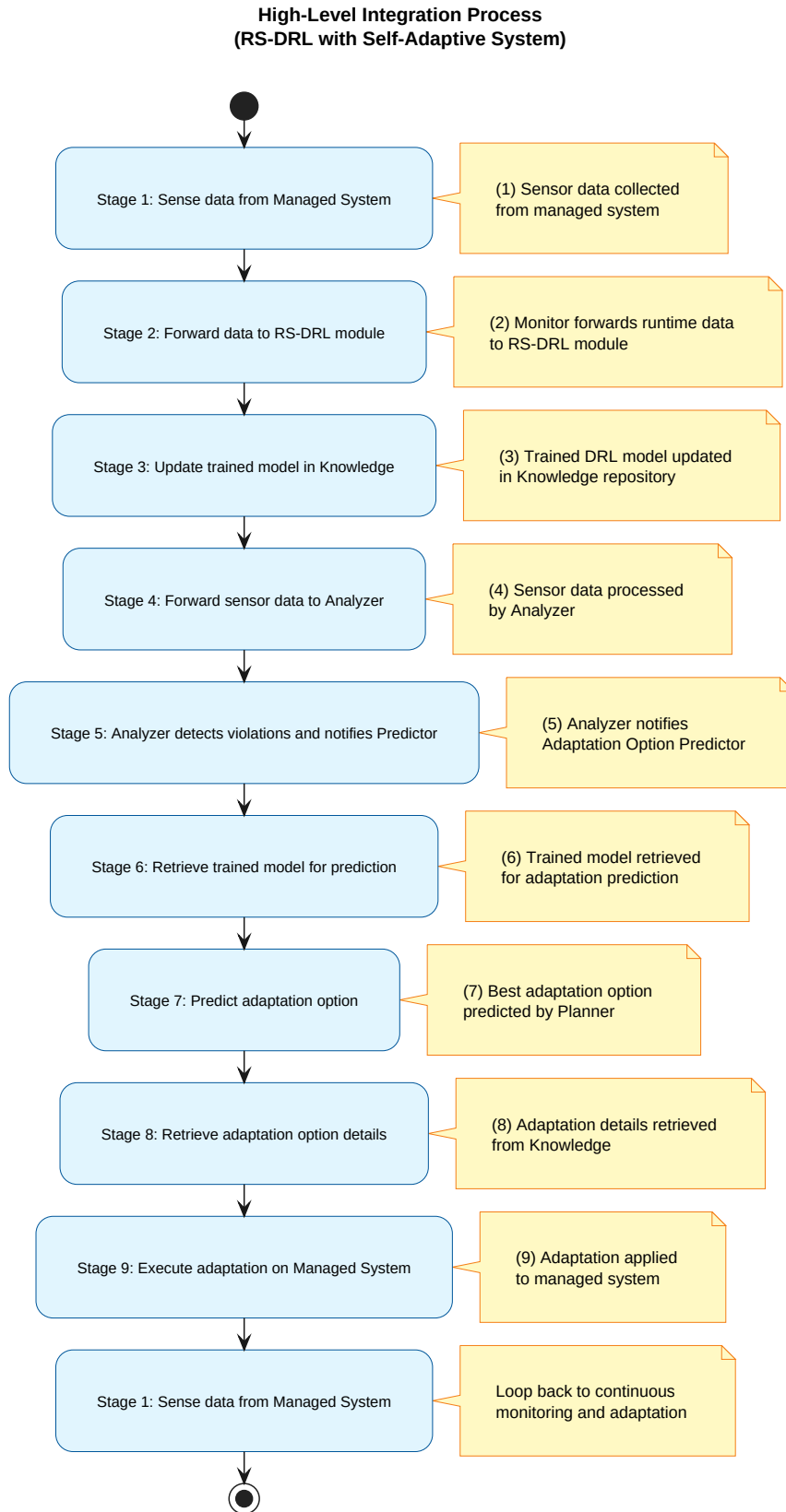


Figure 2: System-level self-adaptation process overview. This figure provides a linear, stage-based walkthrough of the RS-DRL-driven adaptation cycle. It is used throughout this chapter as the primary reference for process-oriented refinement. Stage 3 (asynchronous): Model update occurs independently of the runtime adaptation cycle.

The numbered stages provide traceability throughout the chapter, as each subsequent section references these stages when detailing component processes. All subsequent sections refine **subsets of Stages 1–9**, progressively decomposing the system-level loop into focused subprocesses that answer the question: “What happens next in the system, and why?”

Table 1 classifies the main processes by their temporal nature, providing a high-level view of how different concerns operate within the system.

Process	Nature
Monitoring	Continuous
Prediction	Event-driven
Verification	Conditional
Training	Asynchronous

Table 1: Process classification by temporal nature.

3 Mapping the Architecture to MAPE-K

This section provides a **conceptual mapping** of the RS-DRL architecture to the MAPE-K (Monitor, Analyze, Plan, Execute, Knowledge) architectural pattern, establishing the conceptual bridge between the system-level adaptation loop and the detailed process refinements. This mapping serves as a **classification lens**, not a process description—it categorizes the stages introduced in Section 2 according to MAPE-K functional roles, enabling examiners familiar with MAPE-K to understand how RS-DRL integrates with established self-adaptation patterns. The RS-DRL-based self-adaptive system implements adaptation logic using MAPE-K, enhanced with an RS-DRL module that provides learned intelligence for adaptation decisions.

The RS-DRL module encapsulates the RS-DRL agent described in the original formulation, embedding it within the managing system architecture. In line with the original RS-DRL formulation, the term **adaptation action** refers to the concrete control decision selected by the agent. In this chapter, the synonymous term **adaptation option** is used when emphasizing its architectural realization within the MAPE-K planner.

The mapping of stages to processes (implemented by their respective MAPE-K components) is as follows:

- **Monitor** (Stages 1, 2, 4): The monitoring process observes the managed system, collects runtime data, and forwards state vectors to both the RS-DRL module and the Analyzer.
- **Analyze** (Stage 4): The evaluation process evaluates state vectors against adaptation goals to determine if intervention is required.
- **Plan** (Stages 5, 6, 7): The decision-making process receives adaptation trigger, retrieves trained model, and predicts the most suitable adaptation option.
- **Execute** (Stages 8, 9): The execution process retrieves adaptation option details and applies the adaptation to the managed system.
- **Knowledge** (Stages 3, 6, 8): The Knowledge repository stores trained models, adaptation options, and state history, serving as shared memory for all components.
- **Learning** (Stages 2, 3): The learning process, implemented by the RS-DRL Module, receives state vectors, performs asynchronous training, and stores updated models in the Knowledge repository. Learning operates as a cross-cutting concern that enhances the Plan phase with learned intelligence.

Table 1 (introduced in Section 2) justifies why these processes are separated by their temporal nature, enabling independent processes to be understood and refined separately while maintaining their integration within the overall adaptation process.

The architectural realization of the process illustrated in Figure 2 is shown in Appendix A (Figure A.1), where the RS-DRL-based adaptation loop is mapped onto the MAPE-K reference model.

4 Online Adaptation Decision Process

The following diagrams refine the system-level process overview (Figure 2) by decomposing it into focused subprocesses. Each diagram represents a single responsibility at a consistent abstraction level. This decomposition avoids cognitive overload while preserving traceability through the numbered stages (1–9) and the system-level overview presented in Section 2.

This section details the online adaptation decision process, which represents the execution-time behavior of the system. This process encompasses monitoring, analysis, planning (including adaptation option prediction and selection), verification, and execution. The process is triggered when the system needs to adapt and operates synchronously with the runtime adaptation loop.

4.1 Monitoring and Feature Extraction

The monitoring process begins with the **Sensor Data** component, which collects raw information from the system and its environment. The data preprocessing phase applies filtering, normalization, and missing-value handling to transform raw sensor data into preprocessed data suitable for feature extraction.

The **Feature Extractor** component processes preprocessed data to generate a compact state representation, deriving goal-relevant attributes while reducing dimensionality. Stage 2 provides state information to the RS-DRL module for both runtime inference and asynchronous learning, without implying synchronous training during runtime operation.

The extracted features are assembled into a state vector s_t that serves as a contract between monitoring, learning, and planning. The state vector is validated to ensure that all feature values are within expected bounds, maintaining the interface contract that enables consistent communication between system components. The responsibility of this phase is to transform preprocessed data into a validated state vector. Inputs are preprocessed sensor data; outputs are validated state vectors suitable for use by learning and planning components.

The state vector interface contract is formally defined as follows:

- **Structure:** $s_t \in \mathbb{R}^d$ where d is the dimensionality determined by goal-relevant feature categories
- **Feature categories:** Performance metrics (e.g., latency, throughput), resource utilization (e.g., CPU, memory), and environmental conditions (e.g., load, network state)
- **Stability:** Feature values are normalized to a common range, ensuring consistent scale across different sensor types
- **Contract properties:** All components (Monitor, RS-DRL, Analyzer) operate on the same state representation, maintaining interface consistency

Once constructed, the state vector is disseminated to multiple system components. The state is forwarded to the Analyzer (Stage 4) for goal evaluation, enabling the Analyzer to determine whether adaptation is required. The state is also forwarded to the RS-DRL module (Stage 2) for either learning (during training episodes) or prediction (during runtime adaptation). Additionally, the state history is stored in the Knowledge repository for historical reference and potential future analysis.

Process boundary: This monitoring and state construction process ends once a validated state vector exists and has been disseminated. Everything that follows—decision-making, learning, or execution—operates on this validated state representation, maintaining a clear separation of concerns between perception (monitoring) and action (decision and execution).

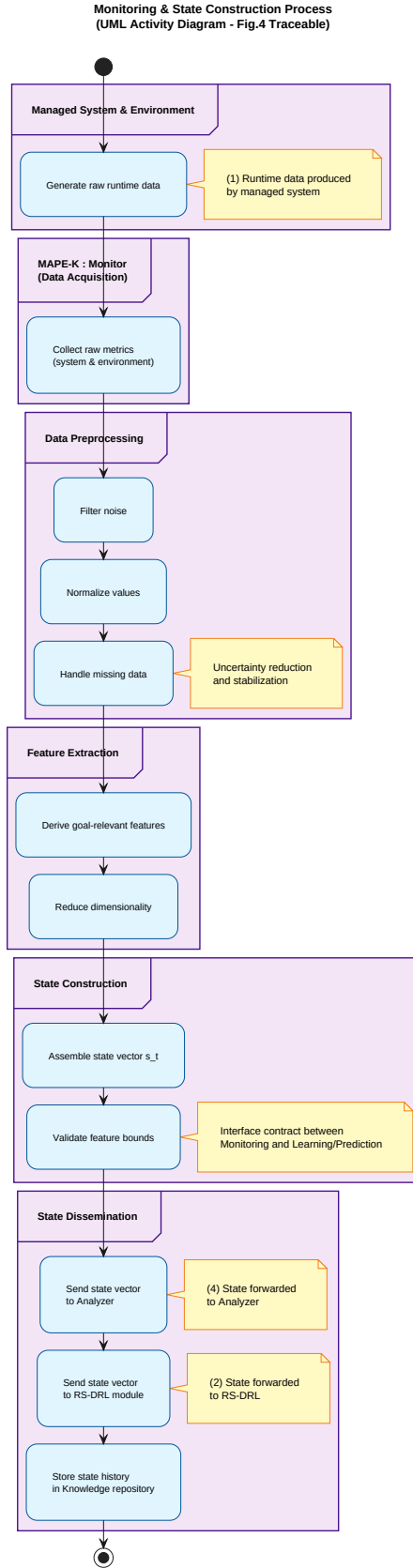


Figure 3: Monitoring and state construction: perception process transforming raw data into decision-ready state vectors. This process starts at Stage 1 (data acquisition) and terminates at Stage 4 (state dissemination to Analyzer) and Stage 2 (state forwarding to RS-DRL module). Raw runtime data is filtered, abstracted, and transformed into a validated state vector suitable for both RS-DRL learning and runtime adaptation analysis.

4.2 Analysis and Context Interpretation

The **Analyzer** component coordinates goal evaluation and adaptation triggering. The Analyzer receives state vectors from the Monitor (Stage 4) and performs goal evaluation, checking whether the current system state satisfies the specified adaptation goals or whether violations or risks are present. If no violation is detected, the system continues normal operation without triggering adaptation. When the Analyzer detects that adaptation goals are violated or are no longer achievable, it notifies the Adaptation Option Predictor, which is embedded in the Planner (Stage 5), triggering the planning phase.

The analysis process validates the state vector against adaptation goals, providing context interpretation that determines whether intervention is required. This validation ensures that adaptation decisions are made only when necessary, preserving system responsiveness during normal operation.

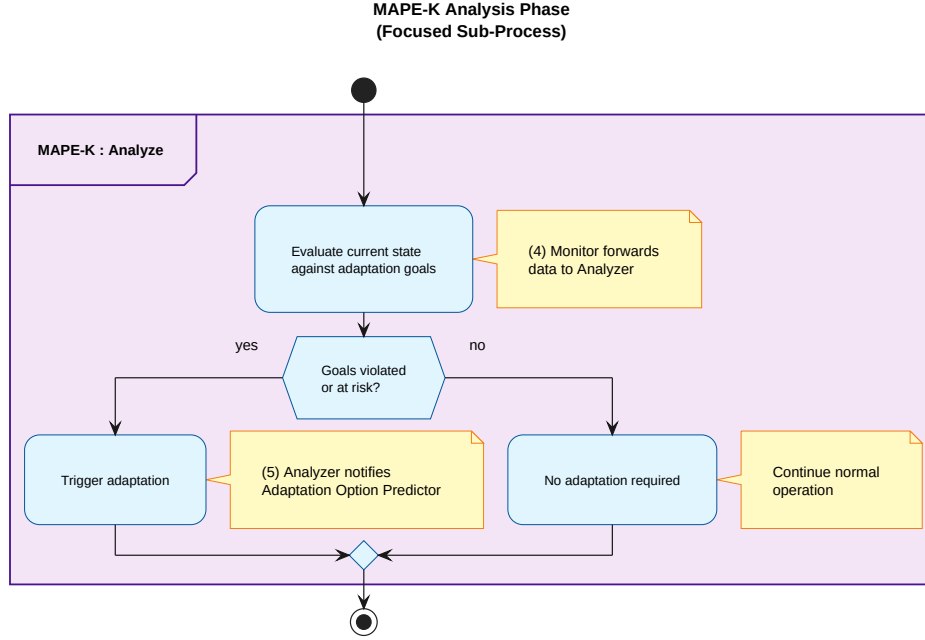


Figure 4: Analysis and adaptation triggering: when and why the system decides to adapt. This process starts at Stage 4 (state reception) and terminates at Stage 5 (notification to Predictor). The Analyzer component evaluates system state against adaptation goals, detects violations or risk conditions, and triggers the planning phase when intervention is required.

4.3 Planning: Adaptation Option Prediction and Selection

The planning process is triggered when the Analyzer detects that adaptation goals are violated or are no longer achievable. Upon this detection, the Analyzer notifies the Adaptation Option Predictor, which is embedded in the Planner (Stage 5). This notification initiates the planning phase of the MAPE-K loop.

Upon receiving the prediction trigger, the Adaptation Option Predictor retrieves the trained RS-DRL model (Stage 6). The predictor receives the current state vector from the monitoring process and evaluates Q-values for all available adaptation options using the trained deep Q-network (DQN).

The DQN estimates Q-values for each action given the current system state, providing a ranking of adaptation options based on their expected long-term value. This evaluation leverages the knowledge learned during training episodes, enabling the system to make informed adaptation decisions without requiring additional learning at runtime.

The Adaptation Option Predictor selects the best adaptation option based on the evaluated Q-values. At runtime, the selection strategy is typically purely exploitative (i.e., $\epsilon \approx 0$), choosing the option with the highest Q-value ($\text{argmax } Q$) rather than exploring alternative options. This exploitation strategy ensures that the system leverages its learned knowledge to make optimal decisions during runtime adaptation.

Process boundary: This planning process is **pure inference**—it uses a pre-trained model to evaluate options but performs no learning, no replay memory sampling, and no policy updates. The ϵ -greedy strategy mentioned here refers only to the selection mechanism during inference, not to training-time exploration. All training logic is strictly isolated to the asynchronous learning process (Section 5).

The predicted adaptation option (Stage 7) completes the planning phase. This option is then forwarded for verification (Section 4.4) and subsequent execution (Section 4.5). This separation ensures that computationally expensive learning activities do not interfere with runtime responsiveness.

Adaptation Option Prediction Process
(Runtime Decision Path - Stages 5-7 traceable to Figure 2)

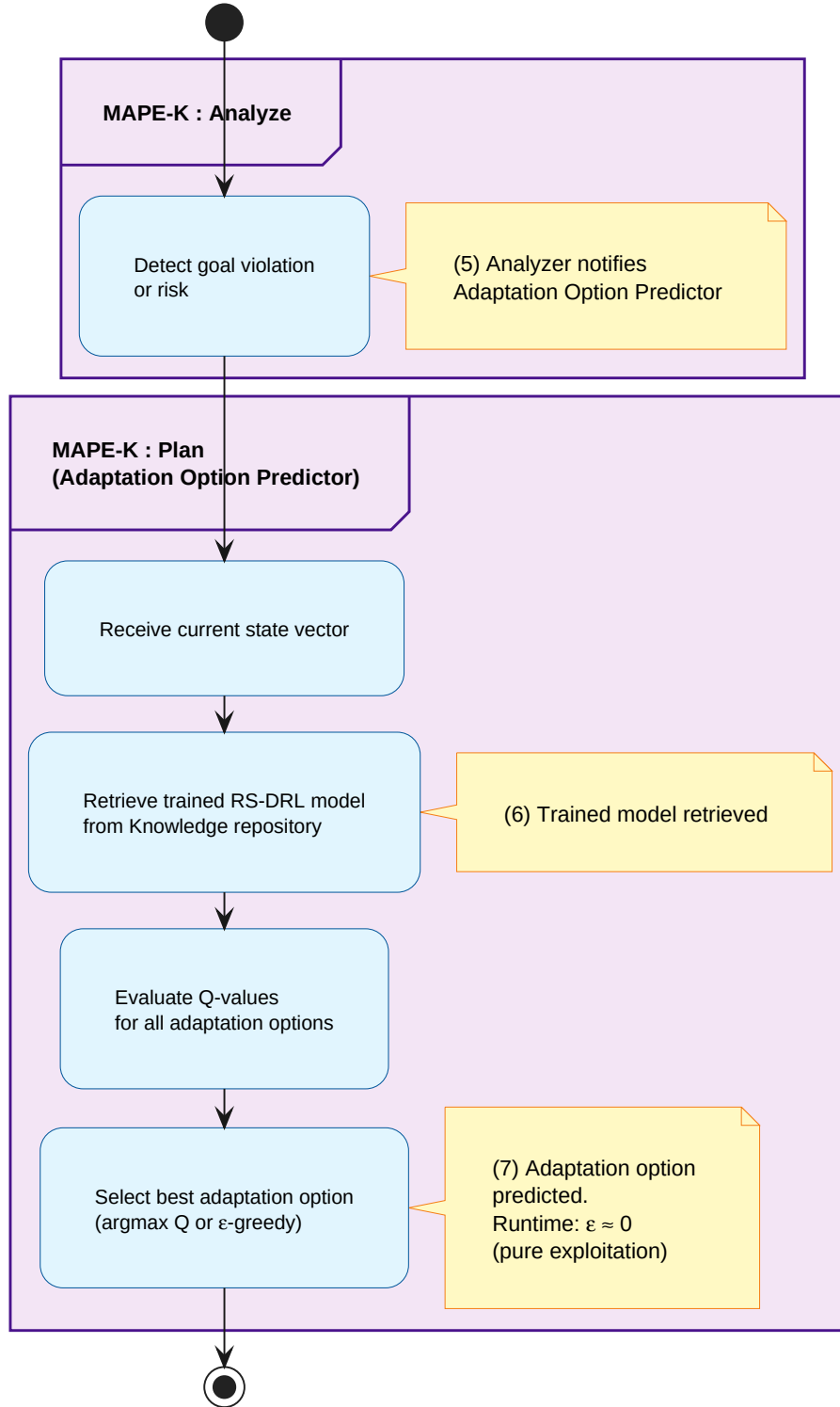


Figure 5: Runtime decision path: adaptation option prediction without learning. This process starts at Stage 5 (Analyzer notification) and terminates at Stage 7 (option prediction). Upon detection of goal violations, the trained RS-DRL model is retrieved (Stage 6) and used to predict the most suitable adaptation option through Q-value evaluation. The adaptation option predicted at Stage 7 is the final output of this process. This figure explicitly excludes training, replay memory sampling, policy updates, and execution (Stage 8) to maintain focus on runtime decision-making.

4.4 Adaptation Option Verification

The verification process provides semantic grounding and trustworthiness for learning-based decisions. Verification belongs to **runtime justification**—its primary purpose during the adaptation decision process is to evaluate predicted options and provide quality estimates that justify adaptation decisions. While verification also generates reward signals that are consumed by the learning process (Section 5), reward generation is a **by-product** of the verification process, not its primary goal during runtime decision-making.

The verification process is triggered when a candidate adaptation option is selected by the Adaptation Option Predictor. The **Adaptation Option Verifier** receives the candidate option along with the current system state, enabling it to evaluate the expected impact of the adaptation.

The verifier retrieves a runtime model that reflects the current system architecture and context. This model is parameterized with the current system state and the candidate adaptation option, creating a specific instance that can be analyzed to predict the adaptation’s effects.

The verification process employs statistical model checking, implemented via UPPAAL-SMC, to estimate the probability of satisfying adaptation goals under uncertainty. The UPPAAL-SMC engine executes probabilistic simulations, exploring different execution paths and environmental conditions to estimate goal satisfaction probabilities.

This statistical approach handles uncertainty by considering multiple possible futures rather than relying on a single deterministic prediction. The approach is particularly valuable in self-adaptive systems, where environmental conditions and system behavior are inherently uncertain.

The UPPAAL-SMC verification process operates under the following assumptions and limitations: (1) the runtime model abstracts system behavior with sufficient fidelity to capture relevant adaptation effects, (2) simulation bounds (number of runs, time horizons) are chosen to balance statistical confidence with computational cost, and (3) the estimated probabilities provide statistical confidence intervals rather than absolute guarantees, requiring interpretation within the context of model abstraction fidelity.

The verification process extracts quality-of-service (QoS) metrics from the simulation results, estimating quality attributes such as energy consumption, packet loss, and latency. During planning, verification provides decision justification and quality estimates. The quality estimates are used to justify adaptation decisions, providing trustworthiness and explainability for learning-based choices. This enables the system to explain why a particular adaptation option was selected, based on predicted quality attributes derived from runtime model analysis.

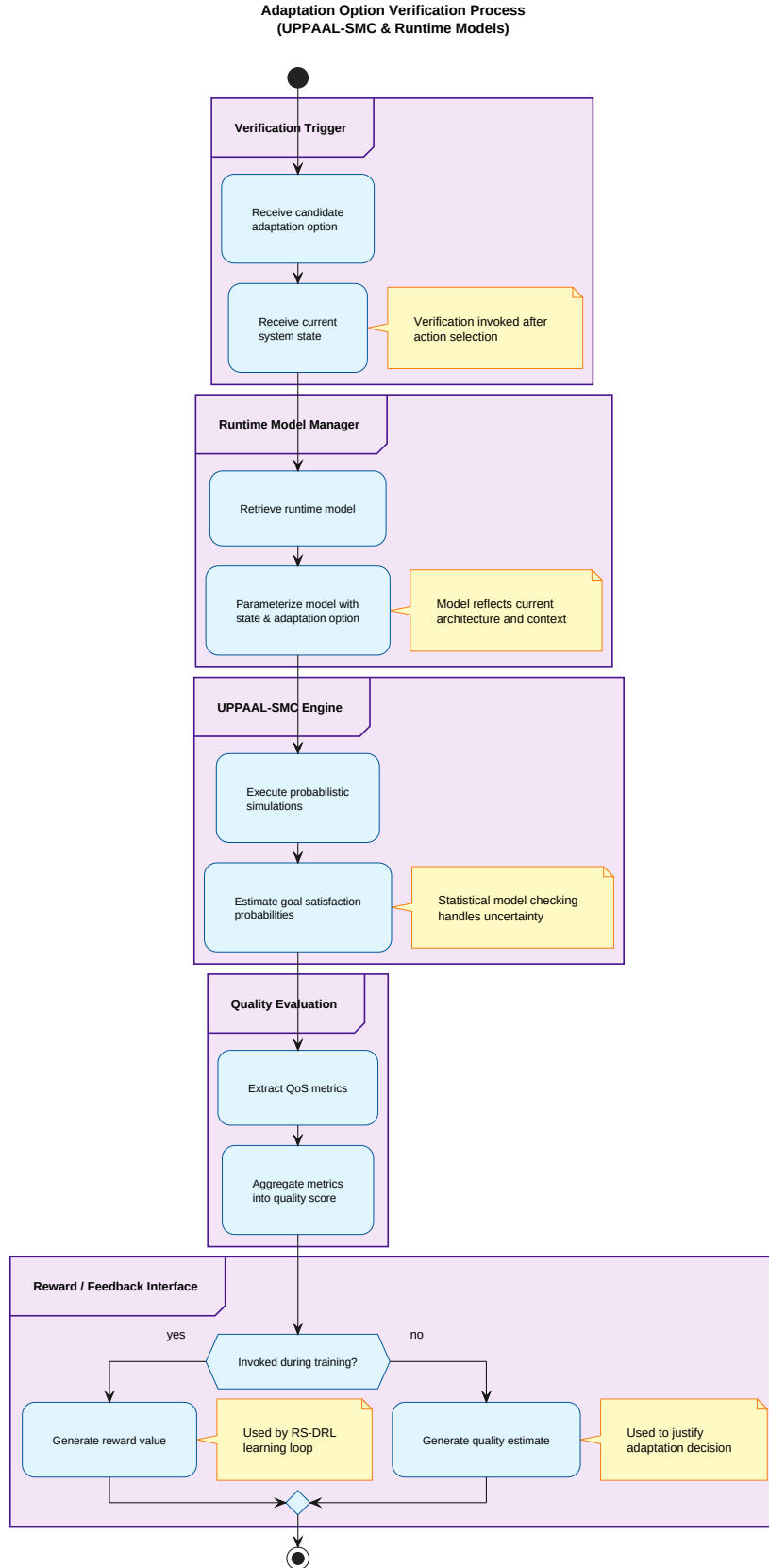


Figure 6: Uncertainty evaluation subprocess: adaptation option verification using statistical model checking. This process validates predicted options under uncertainty, using runtime models and UPPAAL-SMC to estimate goal satisfaction probabilities. The verification process extracts QoS metrics and generates two distinct outputs: **Decision feedback (runtime)** for decision justification during planning, and **Learning reward (asynchronous)** for training during learning, ensuring semantic grounding for learning-based decisions.

4.5 Execution of Adaptation

The **Executor** component coordinates adaptation enactment. The Executor retrieves the selected adaptation option (Stage 8) and applies it to the managed system (Stage 9). The Executor is responsible for enacting the adaptation decision, translating the selected option into concrete system modifications. After adaptation execution, monitoring resumes, and the control loop continues.

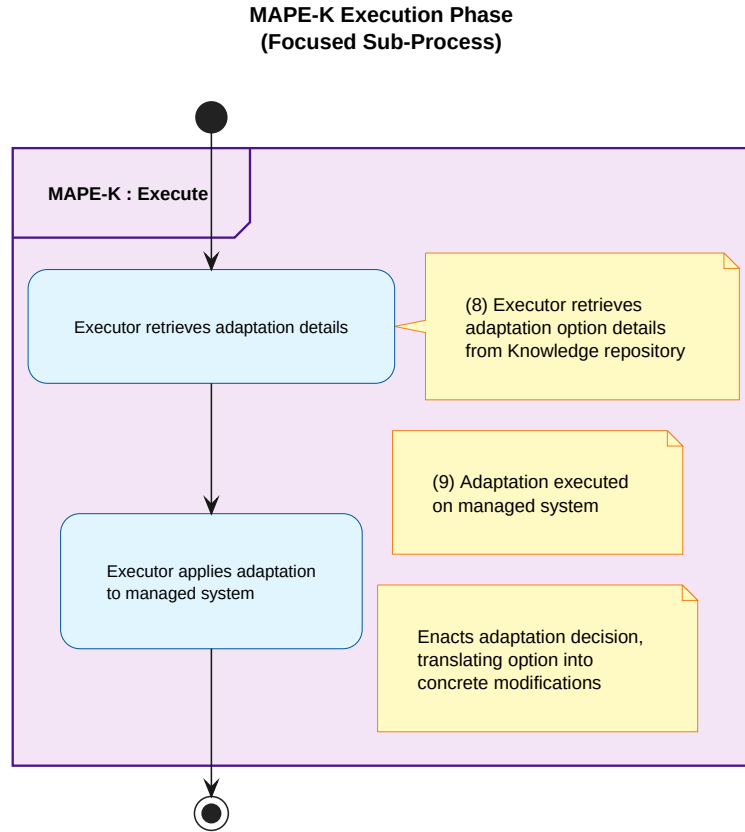


Figure 7: Execution and system reconfiguration: how decisions affect the managed system. This process starts at Stage 8 (option retrieval) and terminates at Stage 9 (adaptation application). The Executor component retrieves and applies the selected adaptation to the managed system, completing the adaptation cycle.

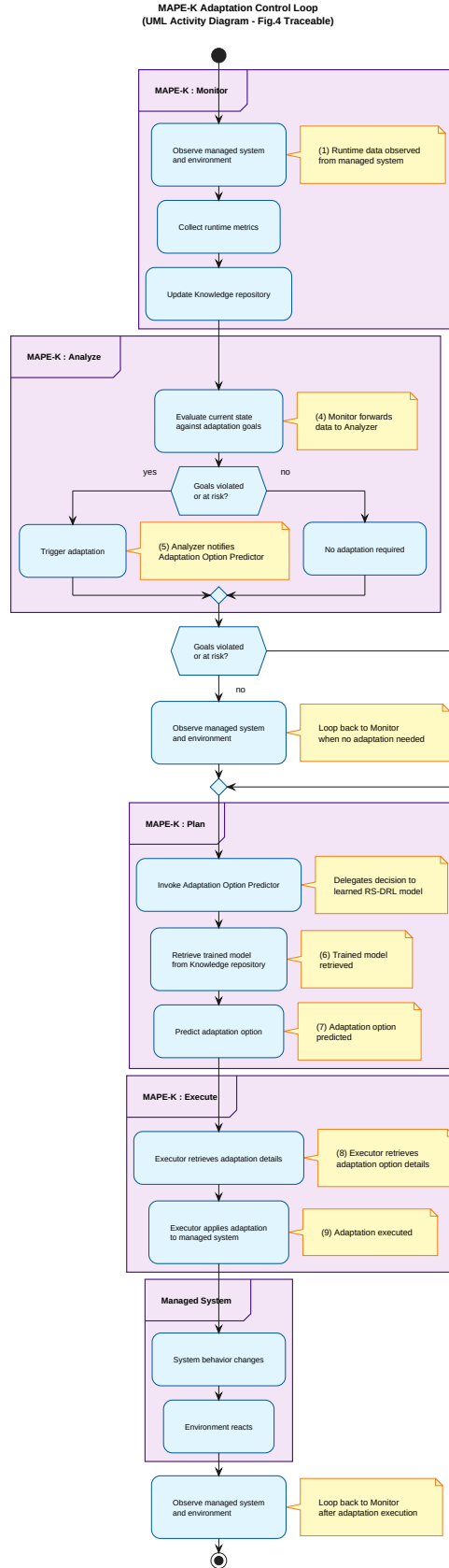


Figure 8: MAPE-K control loop summary: validation view of the adaptation architecture. This optional figure serves as a control-theoretic validation lens, illustrating how monitoring and analysis trigger adaptation, how planning delegates decision making to the RS-DRL predictor, and how execution enacts the selected adaptation. This figure complements the system-level process overview (Figure 2) by providing a MAPE-K-specific perspective for theory-oriented examiners.

5 RS-DRL Internal Decision and Learning Architecture

This section corresponds to the **RS-DRL module** shown in Figure 4 of the original article, which serves as the innovation center of the self-adaptive system. While MAPE-K provides the architectural framework for self-adaptation, the RS-DRL module is what makes adaptation scalable, robust, and uncertainty-aware. The RS-DRL module is realized through the processes described in the following sections: the runtime inference path (Section 4.3 and Section 4.4), feedback generation (Section 7), and the learning path (Section 5).

The RS-DRL module encompasses three interconnected subprocesses: (1) the **runtime inference path**, which includes the Adaptation Option Selector (embedded in the Planner) and the Adaptation Option Verifier, enabling informed decision-making using learned policies; (2) the **feedback generation path**, which includes the Reward Calculator that transforms verification outcomes into reward signals; and (3) the **learning path**, which includes Replay Memory, the novel reward-shaping mechanism, and Model Training, enabling continuous policy improvement. These subprocesses collectively implement the RS-DRL module’s core functionality, with the reward-shaping mechanism representing the key innovation that accelerates learning and promotes generalization across heterogeneous environments.

6 RS-DRL Learning and Policy Improvement (Reward-Shaping Core)

This section details the learning and policy improvement process, which operates asynchronously in the background to refine the DRL model. This process is triggered by runtime episodes but operates independently of the online adaptation decision process, enabling continuous learning without interfering with runtime responsiveness. This section focuses on **learning consumption of feedback**—how the learning process uses verification-derived rewards and experience tuples to improve adaptation policies.

6.1 Reward Calculation

During learning, the learning process consumes verification-derived reward signals that were generated during the verification phase (Section 4.4). The **Reward Calculator** assigns rewards based on the verifier’s estimates, thereby guiding the learning process of the RS-DRL agent. These verification-derived reward signals are used to update the Q-network, enabling the agent to learn which adaptation actions lead to desirable outcomes. By mediating between architectural models and learning, verification ensures that rewards remain aligned with system-level objectives, even as the learning process adapts to new experiences.

The verification process extracts quality-of-service (QoS) metrics from simulation results, estimating quality attributes such as energy consumption, packet loss, and latency. These metrics are aggregated as reward values during learning, providing feedback signals that guide the learning process.

6.2 Experience Collection and Replay Memory

Training proceeds in episodes, with each episode representing a complete interaction cycle between the RS-DRL agent and the environment. At the beginning of each episode, the training process is initialized, and the current state s_t is received from the monitoring process (Stage 2). This state vector serves as the input to the learning process, providing a compact representation of the current system condition and environment context.

The **Adaptation Action Selector** identifies candidate adaptation actions using an epsilon-greedy policy, balancing exploration and exploitation. During early training, the agent explores the action space to discover effective adaptation strategies, while gradually shifting toward exploitation of learned knowledge as training progresses. This balance is critical for effective learning in uncertain environments.

The epsilon-greedy policy selects actions probabilistically: with probability ε , a random action is chosen (exploration), and with probability $1 - \varepsilon$, the action with the highest Q-value is chosen (exploitation). The exploration rate ε decays over time, ensuring that the agent transitions from exploration to exploitation as it learns.

The selected action is executed in the environment, producing a reward r_t and a subsequent state s_{t+1} . This interaction generates the experience tuple $\langle s_t, a_t, r_t, s_{t+1} \rangle$ that will be used for learning.

The **Replay Memory** component stores experience tuples $\langle s_t, a_t, r_t, s_{t+1} \rangle$ as they are generated during environment interaction. Unlike online learning approaches that update the model immediately after each

experience, experience replay decouples data generation from learning, enabling more stable and sample-efficient training.

The replay memory maintains a buffer of past experiences, allowing the learning process to sample minibatches from a diverse set of experiences rather than relying solely on recent, potentially correlated experiences. This diversity helps break the correlation between consecutive experiences and enables the learning algorithm to learn from rare but important events that may have occurred earlier in training.

Learning is triggered only when the replay buffer contains sufficient experiences, ensuring that the model updates are based on a representative sample of the experience distribution. This gating mechanism prevents premature learning from insufficient or biased experience samples.

6.3 Model Training and Policy Update

When the replay buffer is sufficiently populated, minibatches are sampled for learning. The **Replay Memory** incorporates a novel reward-shaping mechanism. Unlike conventional experience replay, RS-DRL selectively reshapes certain failed experiences and treats them as successful, mimicking aspects of human learning. This mechanism (1) promotes generalization across heterogeneous environments and (2) accelerates learning by transforming past mistakes into informative training samples.

After sampling a minibatch from the replay buffer, each experience is evaluated to determine whether it represents a failure transition (i.e., $r = 0$). Failed experiences are then probabilistically reinterpreted as optimistic successes according to a reshaping probability p . Importantly, reward reshaping is applied only to the sampled minibatch and does not alter the contents of the replay buffer. This design preserves the diversity and integrity of accumulated experiences while mitigating the dominance of failure transitions during early learning stages, thereby accelerating convergence without biasing long-term policy optimization.

During **Model Training**, batches of experiences (with reshaped rewards) are used to update the DQN parameters via backpropagation to minimize the loss function, progressively improving Q-value estimation. The training process computes temporal difference (TD) targets using the reshaped rewards, computes the loss between predicted and target Q-values, and performs gradient descent to update the Q-network parameters.

The TD target computation uses the Bellman equation: $y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$, where γ is the discount factor, s' is the next state, and θ^- represents the parameters of a target network. The target network provides stable target values for TD learning, preventing the moving target problem that can destabilize learning.

The training process also includes periodic updates to the target network, which copies the current Q-network parameters to the target network at regular intervals. Additionally, exploration rate decay gradually shifts the policy from exploration to exploitation as training progresses, ensuring that the agent transitions from learning to applying learned knowledge.

The trained model is stored in the **Trained Models Repository**, which is part of the Knowledge repository. This storage (Stage 3) makes the updated model available for use by the Adaptation Option Predictor during runtime adaptation episodes. The model storage ensures that the latest learned knowledge is preserved and accessible for runtime decision making.

The repository management maintains versioning and access control for trained models, enabling the system to track model evolution over time and select appropriate models for different contexts or requirements. This management is crucial for maintaining system reliability and enabling rollback to previous model versions if needed.

Although learning proceeds episodically, the RS-DRL process remains *continuous* in the sense that model updates are repeatedly integrated into ongoing system operation, as defined in the original approach.

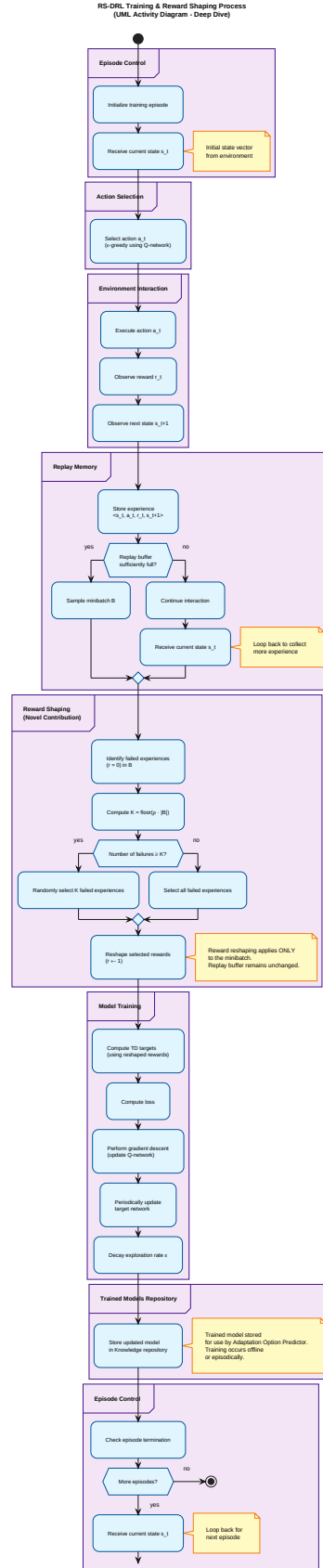


Figure 9: Asynchronous learning and policy update: RS-DRL training and reward shaping process. This is the only figure allowed to be dense, as learning is inherently algorithmic and complex. The figure details episodic interaction with the environment, experience replay, and the novel reward reshaping mechanism applied during minibatch learning. The process encompasses episode initialization, action selection, environment interaction, experience storage, replay buffer management, minibatch sampling, reward shaping, Q-network training, and model storage (Stage 3).

7 Knowledge Management in RS-DRL Architecture

This section explains knowledge management in the RS-DRL architecture from a **coordination semantics** perspective, focusing on **who reads what and when**, rather than re-explaining storage mechanics that have already been described in previous sections. The Knowledge element of MAPE-K is critical for maintaining system coherence and enabling coordination between independent processes with different temporal natures.

The **Knowledge Repository** serves as the central coordination point, storing:

- **Trained DRL Model** (updated at Stage 3, read at Stage 6)
- **Adaptation Options** (read at Stage 8)
- **State History** (written during monitoring, available for analysis)
- **Replay Memory** (written during learning, read during training)

Knowledge coordination occurs through well-defined read-write contracts:

- **When monitoring completes** (Stage 1): State vectors are stored in Knowledge for historical reference.
- **When learning completes** (Stage 3): Trained models are written to Knowledge, making learned policies available for future runtime use.
- **When planning begins** (Stage 6): Trained models are read from Knowledge, enabling the Adaptation Option Predictor to leverage learned knowledge.
- **When execution begins** (Stage 8): Adaptation option details are read from Knowledge, providing the Executor with the specifications needed to enact adaptations.

This coordination structure ensures that all components operate on consistent information while maintaining clear separation of concerns. The Knowledge repository enables independent processes with different temporal natures (continuous monitoring, event-driven prediction, asynchronous learning) to synchronize effectively without tight coupling.

8 RS-DRL Feedback and Reward Calculation Process

This section discusses the critical interaction between the online adaptation decision process and the asynchronous learning process, explaining how temporal decoupling enables both stability and adaptability in the RS-DRL system.

8.1 Temporal Decoupling

The online adaptation decision process (Section 4) and the learning process (Section 5) operate with different temporal characteristics. The decision process is event-driven and synchronous with the runtime adaptation loop, responding immediately when adaptation is required. The learning process operates asynchronously, triggered by runtime episodes but executing independently in the background.

This temporal decoupling is essential for system stability. By separating learning from decision-making, the system ensures that computationally expensive training activities do not interfere with runtime responsiveness. The learning process can refine policies over multiple episodes without disrupting the system's ability to make timely adaptation decisions.

8.2 Stability vs Adaptability

The interaction between decision-making and learning creates a balance between stability and adaptability. The online decision process provides stability by using a trained model that has been refined through multiple learning episodes. This stable model enables consistent, reliable adaptation decisions during runtime.

The learning process provides adaptability by continuously refining the model based on new experiences. As the system encounters new situations and environmental conditions, the learning process updates the

model to improve future decision-making. This adaptability ensures that the system can evolve its behavior to handle changing conditions and new challenges.

The balance between stability and adaptability is maintained through the Knowledge repository, which stores the trained model and makes it available to the decision process. The model is updated asynchronously, ensuring that the decision process always has access to a stable, trained model while the learning process continues to refine it in the background.

8.3 Why RS-DRL Supports Uncertainty

The interaction between decision-making and learning enables RS-DRL to effectively handle uncertainty in self-adaptive systems. The learning process accumulates experience over time, building a model that captures patterns and relationships in uncertain environments. The decision process leverages this learned knowledge to make informed decisions even when facing uncertain conditions.

The temporal decoupling allows the system to learn from both successful and unsuccessful adaptations, building a comprehensive understanding of the adaptation space. The reward shaping mechanism (Section 5.3) further enhances this by transforming failed experiences into learning opportunities, accelerating the system’s ability to adapt to uncertain conditions.

The verification process (Section 4.4) provides additional support for uncertainty by using statistical model checking to evaluate adaptation options under uncertainty. This verification generates both quality estimates for decision justification and reward signals for learning, ensuring that both decision-making and learning remain aligned with system-level objectives even in uncertain environments.

9 Summary of Architectural Processes

This section synthesizes the process model and provides a summary of the architectural processes presented in this chapter.

9.1 Process Model Summary

This chapter presented a structured process model of the RS-DRL-based self-adaptive system, organized through progressive refinement from a system-level adaptation loop (Section 2) to detailed process refinements (Sections 4 through 7). The chapter defines **the RS-DRL adaptation process** as the dominant end-to-end process that continuously monitors system state, evaluates adaptation needs, makes hierarchical decisions, executes adaptations, and learns from feedback under uncertainty.

9.2 Process Hierarchy and Decomposition

The RS-DRL adaptation process decomposes hierarchically into major process layers:

1. **System-Level Self-Adaptation Loop** (Section 2): The black-box view of continuous adaptation under uncertainty.
2. **MAPE-K Mapping** (Section 3): The conceptual bridge mapping processes to MAPE-K components.
3. **Online Adaptation Decision Process** (Section 4): Execution-time behavior encompassing monitoring, analysis, planning, verification, and execution.
4. **Learning and Policy Improvement Process** (Section 5): Asynchronous background learning that refines adaptation policies.
5. **Knowledge Management** (Section 6): Central coordination through shared knowledge repository.
6. **Interaction Between Processes** (Section 7): Temporal decoupling and coordination mechanisms.

This hierarchical decomposition enables each process to be understood and refined independently while maintaining their integration within the overall adaptation loop. The nine numbered stages (1–9) provide traceability throughout the chapter, with each stage traceable to specific activities in the component process diagrams.

9.3 Uncertainty Management Through Process Organization

This process-oriented organization supports uncertainty management by: (1) separating temporal concerns (continuous monitoring, event-driven prediction, asynchronous learning, conditional verification) as classified in Table 1, enabling each process to handle uncertainty at its appropriate abstraction level; (2) maintaining clear interfaces (state vectors, Knowledge repository) that enable processes to coordinate without tight coupling; and (3) providing semantic grounding through the uncertainty evaluation process that ensures learning and decision-making remain aligned with system-level objectives. The process model refines the reference architecture (Figure 4), making explicit the control flow, data flow, and feedback loops that enable continuous adaptation under uncertainty.

The chapter prepares the ground for evaluation and experiments (Chapter 6) by establishing a clear process-oriented foundation that enables systematic analysis of the RS-DRL approach’s effectiveness in managing uncertainty in self-adaptive systems.